@codedsoul_05

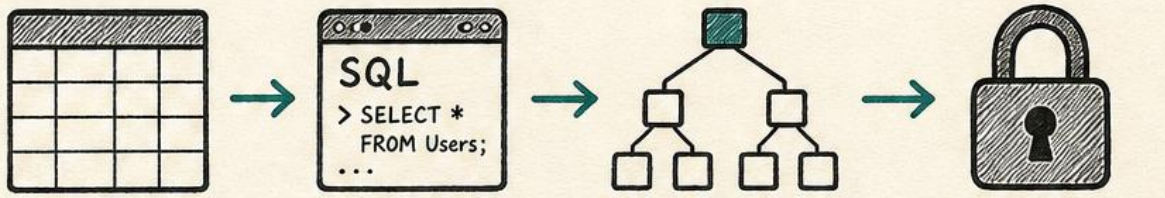
DBMS NOTES v2.0 ★

DBMS

BEGINNER → ADVANCED

COMPLETE PREMIUM NOTES

From ER diagrams to transactions — everything you need to design, query, and run databases with confidence.



SCHEMA → QUERY → INDEX → TRANSACTION

"Data is an asset. Consistency is a promise."

ER • SQL • Keys • Normalization • Indexes • ACID • Concurrency • NoSQL

1 WHAT'S INSIDE

- Fundamentals
- ER Model
- Relational Model
- Keys
- Normalization
- SQL Basics
- Joins
- Advanced SQL
- Indexes
- Transactions
- Concurrency
- Storage
- Optimization
- NoSQL
- Security
- Recap

2 BY THE NUMBERS

- 40+ Topics
- 60+ Queries
- ACID Mastery
- Beginner to Pro
- Interview Ready
- Design Recipes

3 TOPICS COVERED

- ☑ ER & Schema Design
- ☑ Relational Algebra
- ☑ Normalization 1NF–BCNF
- ☑ SQL DDL/DML
- ☑ Joins & Subqueries
- ☑ Indexes & Plans
- ☑ ACID & Isolation
- ☑ Concurrency Control
- ☑ Backup & Security
- ☑ NoSQL & CAP

★ Design clean. Query smart. Keep consistent. ★

01 — WHAT & WHY DBMS

GOAL
Store, organize, and retrieve data safely at scale.

THE PROBLEM
File systems — redundancy, inconsistency, no concurrency, weak security.

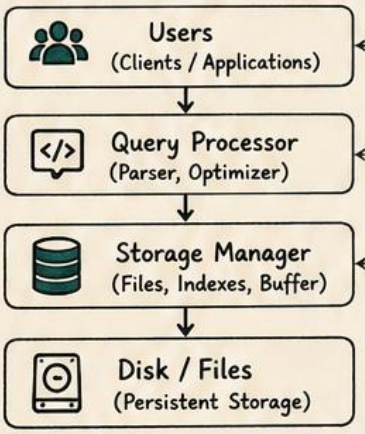
DBMS BENEFITS

- Data independence
- Integrity
- Concurrent access
- Recovery
- Security
- Query language (SQL)

DBMS vs FILE SYSTEM

FEATURE	DBMS	FILE SYSTEM
Data Redundancy	Minimized	High
Consistency	Enforced (Constraints)	Hard to maintain
Concurrent Access	Yes (Multi-user)	No / Limited
Data Independence	High (Logical & Physical)	Low (Tight coupling)
Security	Strong (Users, Roles, Privileges)	Weak (File-level)
Recovery	ACID, Logs, Backup	Manual, Limited
Query Capability	Powerful (SQL)	Limited / None
Data Relationships	Supported (Keys, Relations)	Not supported
Maintenance	Centralized	Application dependent

ARCHITECTURE (HIGH LEVEL)



DBA
Designs, secures, and manages the database.

APPLICATION
Builds features using DBMS.

END USER
Interacts with application data.

WHERE DBMS SHINES

BANKING
Transactions, accounts, compliance.

E-COMMERCE
Orders, inventory, payments, customers.

ANALYTICS
Large datasets, reporting, insights.

SAAS
Multi-tenant data, scalability.

MOBILE BACKENDS
Real-time data sync, user services.

TIP: DBMS manages data + access rules + recovery — not just files.

★ One source of truth for shared data. ★

02 — DATA MODELS & ER

GOAL: Model real-world data before writing SQL.

ENTITY-RELATIONSHIP

ENTITY

- A real-world object or thing.
- Ex: Student, Course, Dept.

ATTRIBUTE

- key
- composite
- multivalued
- derived

RELATIONSHIP

- 1 : 1
- 1 : N
- M : N

CARDINALITY

1 : 1

1 : N

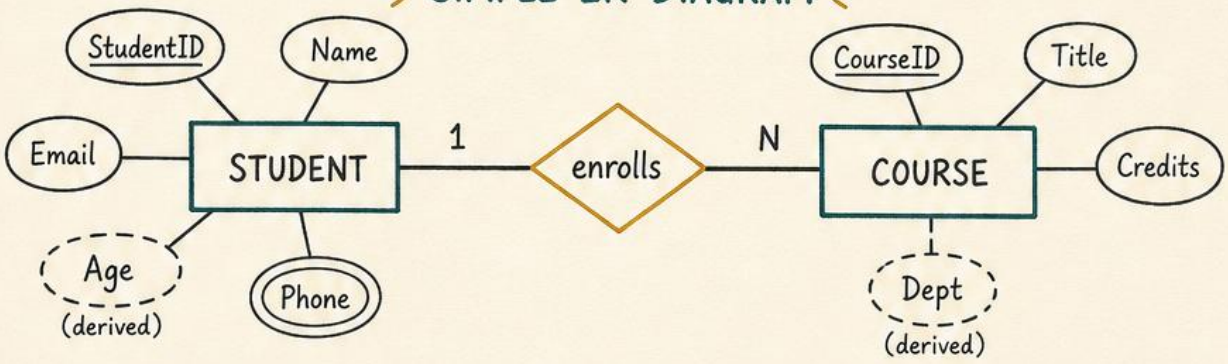
M : N

PARTICIPATION

(total)

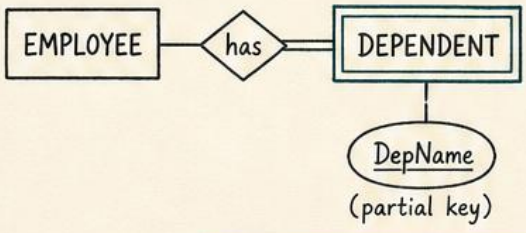
(partial)

SIMPLE ER DIAGRAM



WEAK ENTITY

- Does not have a primary key of its own.
- Identified by its owner entity + partial key.
- Represented by double rectangle.
- Total participation in identifying relationship.



ER TO RELATIONAL MAPPING

- ☑ **Strong entity** → Create a table with its attributes. Primary key = key attribute.
- ☑ **Weak entity** → Create a table with its attributes + owner's primary key. Primary key = (owner PK + partial key).
- ☑ **1 : 1 relationship** → Add foreign key on one side (prefer total participation side).
- ☑ **1 : N relationship** → Add foreign key on N side referencing 1 side's primary key.
- ☑ **M : N relationship** → Create a new table with foreign keys from both sides.

Draw the ER first. Tables come second.

03 — RELATIONAL MODEL

GOAL

Represent real-world data in simple, well-structured tables and operate on them using relational algebra.

- **TABLE / RELATION**: A named collection of tuples with the same set of attributes.
- **TUPLE / ROW**: A single record (ordered values for attributes).
- **ATTRIBUTE / COLUMN**: A property of the relation.
- **DOMAIN**: The set of valid values an attribute can take.

RELATION = TABLE

ATTRIBUTES (COLUMNS)

STUDENT

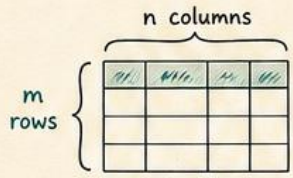
SID (PK)	NAME	DEPT	CGPA
101	Aarav	CSE	8.7
102	Diya	ECE	9.1
103	Kabir	ME	7.8
104	Meera	CSE	9.3
⋮	⋮	⋮	⋮

TUPLES (ROWS)

DEGREE vs CARDINALITY

DEGREE (n): Number of attributes (columns) in a relation. — n

CARDINALITY (m): Number of tuples (rows) in a relation. — m



DOMAIN CONSTRAINTS

- Each attribute has a domain.
- Values must come from its domain.
- Constraints: Type, Range, Format, Uniqueness, CHECK, DEFAULT, NOT NULL, etc.

Example:
- CGPA: 0.0 to 10.0
- SID: Integer, > 0
- NAME: String (not empty)

NULL — WHAT DOES IT MEAN?

NULL represents:
- Unknown value
- Not applicable
- Value not yet available

NULL is not the same as 0, empty string or FALSE.

Use IS NULL / IS NOT NULL to test for NULL.

RELATIONAL ALGEBRA (INTRO) — CORE OPERATIONS

1) SELECT (σ)

Select tuples (rows) that satisfy a condition.

$\sigma_{DEPT='CSE'}(STUDENT)$

SID	NAME	DEPT	CGPA
101	Aarav	CSE	8.7
104	Meera	CSE	9.3
⋮			

2) PROJECT (π)

Select attributes (columns).

$\pi_{SID, NAME}(STUDENT)$

SID	NAME
101	Aarav
102	Diya
103	Kabir
104	Meera

3) JOIN (⋈)

Combine tuples from related tables.

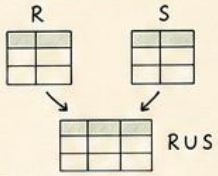
$STUDENT \bowtie_{DEPT=DEPT} DEPARTMENT$

SID	NAME	DEPT	DNAME
101	Aarav	CSE	CS Engg
102	Diya	ECE	ECE
103	Kabir	ME	Mech
104	Meera	CSE	CS Engg
⋮			

4) UNION (∪)

Tuples that are in either relation.

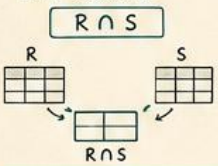
$R \cup S$



5) INTERSECT (∩)

Tuples that are in both relations.

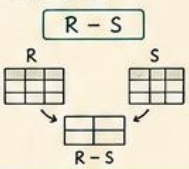
$R \cap S$



6) DIFFERENCE (−)

Tuples in R but not in S.

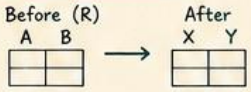
$R - S$



7) RENAME (ρ)

Rename relation or attributes.

$\rho_{NEW(A,B,...)}(R)$



NOTES

- All operations return relations.
- Schema compatibility required for ∪, ∩, −.
- Results are sets (duplicates removed).

INTEGRITY CONSTRAINTS

1) ENTITY INTEGRITY

- Primary key (PK) values must be UNIQUE and NOT NULL.
- No two rows can have the same PK.

2) REFERENTIAL INTEGRITY

- Foreign key (FK) value must match a primary key in the referenced table.
- Prevents orphan / invalid references.

Example:
STUDENT.DEPT → DEPARTMENT.DEPT
Every DEPT in STUDENT must exist in DEPARTMENT.

☆ Relations are sets of tuples — order doesn't matter. ☆

04 — KEYS & CONSTRAINTS

KEY TYPES



PRIMARY KEY

Uniquely identifies each row in a table. No NULL allowed.



CANDIDATE KEY

A minimal super key. Any of these can be chosen as PK.



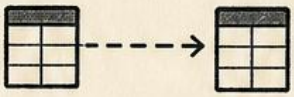
SUPER KEY

Any set of one or more attributes that can uniquely identify a row.



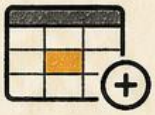
ALTERNATE KEY

Candidate keys not chosen as the primary key.



FOREIGN KEY

References the primary key of another table. Enforces relationships.



COMPOSITE KEY

A key made up of two or more attributes together.



SURROGATE KEY

System-generated key with no business meaning (e.g., auto-increment ID).

CONSTRAINTS



NOT NULL

Column cannot have NULL values.



UNIQUE

All values in the column must be unique.



CHECK

Ensures values satisfy a specific condition.



DEFAULT

Automatically assigns a default value.



PRIMARY KEY

Ensures uniqueness and NOT NULL for the column(s).



FOREIGN KEY

Maintains referential integrity with actions:

- ON DELETE CASCADE
- ON DELETE SET NULL
- ON DELETE RESTRICT

EXAMPLE

STUDENT	
sid	PK
name	
email	
dob	

ENROLL	
sid	FK
cid	FK
grade	



TIP

Prefer stable natural keys carefully; surrogates when natural keys change.

Keys protect identity. FKs protect relationships.

05 — NORMALIZATION

KEY PAGE

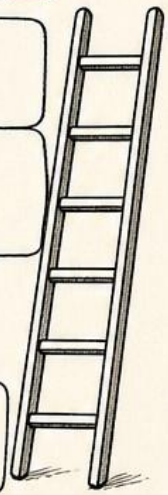
GOAL: Remove redundancy, avoid update / insert / delete anomalies.

FUNCTIONAL DEPENDENCY

A → B

A → B means the value of A determines the value of B.

NORMAL FORMS LADDER



- **1NF** — Atomic values, no repeating groups.
- **2NF** — 1NF + no partial dependency on composite PK.
- **3NF** — 2NF + no transitive dependency.
- **BCNF** — For every FD X → Y, X is a superkey.

EXAMPLE — STUDENT ENROLLMENT

BEFORE (UNNORMALIZED)

ENROLLMENT (StudentID, StudentName, CourseID, CourseName, Instructor, InstructorEmail)

StudentID	StudentName	CourseID	CourseName	Instructor	InstructorEmail
101	Alice	CS101	DBMS	Dr. Rao	rao@uni.edu
101	Alice	CS102	OS	Dr. Mehta	mehta@uni.edu
…	…	…	…	…	…

ANOMALIES

- **UPDATE:** Change Dr. Rao email → must update multiple rows.
- **INSERT:** Cannot insert new course unless a student enrolls.
- **DELETE:** Deleting last student in a course may delete course / instructor info.

AFTER (DECOMPOSED)

STUDENT (StudentID, StudentName)

StudentID	StudentName
101	Alice
…	…

COURSE (CourseID, CourseName, InstructorID)

CourseID	CourseName	InstructorID
CS101	DBMS	1
CS102	OS	2
…	…	…

INSTRUCTOR (InstructorID, Instructor, InstructorEmail)

InstructorID	Instructor	InstructorEmail
1	Dr. Rao	rao@uni.edu
2	Dr. Mehta	mehta@uni.edu
…	…	…

ENROLLMENT (StudentID, CourseID)

StudentID	CourseID
101	CS101
101	CS102
…	…

Redundancy removed. Anomalies avoided.

TIP: Normalize for OLTP (write-heavy, data integrity). Controlled denormalize for read-heavy analytics.

Normalize to reduce anomalies — not for vanity.

06 – SQL BASICS (DDL & DML)

DDL (Data Definition Language)

CREATE TABLE



```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  age INT,
  email VARCHAR(100)
);
```

ALTER TABLE



```
ALTER TABLE students
ADD column phone
 VARCHAR(15);

ALTER TABLE students
MODIFY column age
 INT;
```

DROP TABLE



```
DROP TABLE
  students;
```

CREATE INDEX



```
CREATE INDEX idx_name
ON students(name);
```

DML (Data Manipulation Language)

INSERT



```
INSERT INTO students
(id, name, age, email)
VALUES (1, 'Aman', 20,
'aman@email.com');
```

SELECT



```
SELECT id, name, age
FROM students
WHERE age > 18;
```

UPDATE



```
UPDATE students
SET age = 21
WHERE id = 1;
```

DELETE



```
DELETE FROM students
WHERE id = 1;
```

SQL CLAUSES

SELECT	FROM	WHERE	GROUP BY	HAVING	ORDER BY	LIMIT
Choose columns	From table(s)	Filter rows	Group rows	Filter groups	Sort result	Limit rows

COMMON CONDITIONS & OPERATORS

ALIASES

```
SELECT name AS
student_name
FROM students;
```

DISTINCT

```
SELECT DISTINCT
city FROM
students;
```

BETWEEN

```
SELECT * FROM
students WHERE
age BETWEEN 18
AND 25;
```

IN

```
SELECT * FROM
students WHERE
id IN (1, 2, 3);
```

LIKE

```
SELECT * FROM
students WHERE
name LIKE 'A%';
```

IS NULL

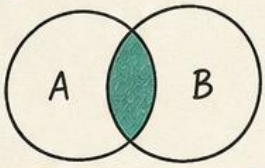
```
SELECT * FROM
students WHERE
email IS NULL;
```

DDL defines structure. DML changes data.

07 – JOINS

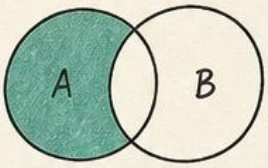
(A) = STUDENT
(B) = ENROLL / COURSE

① INNER JOIN



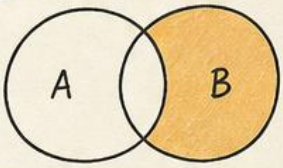

```
SELECT *
FROM STUDENT s
INNER JOIN ENROLL e
 ON s.sid = e.sid;
```

② LEFT JOIN



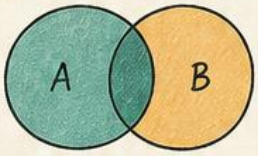

```
SELECT *
FROM STUDENT s
LEFT JOIN ENROLL e
 ON s.sid = e.sid;
```

③ RIGHT JOIN




```
SELECT *
FROM STUDENT s
RIGHT JOIN ENROLL e
 ON s.sid = e.sid;
```

④ FULL OUTER JOIN




```
SELECT *
FROM STUDENT s
FULL OUTER JOIN ENROLL e
ON s.sid = e.sid;
```

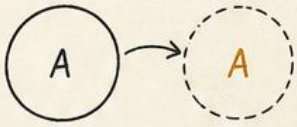
⑤ CROSS JOIN




```
SELECT *
FROM STUDENT s
CROSS JOIN ENROLL e;
```

☆ Every row of A with every row of B

⑥ SELF JOIN




```
SELECT  s1.name, s2.name AS friend
FROM STUDENT s1
JOIN STUDENT s2
 ON s1.dept = s2.dept
  AND s1.sid <> s2.sid;
```

☆ Join a table to itself.

ON vs USING

ON — Use with any condition (including different column names, expressions).

```
... JOIN ENROLL e
ON s.sid = e.sid AND e.grade = 'A';
```

USING — Use when column names are the same in both tables.

```
... JOIN ENROLL e
USING (sid);
```

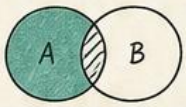
☆ Column appears once in result.

JOIN FILTER: ON vs WHERE

Key difference for OUTER JOINS

Filter in ON

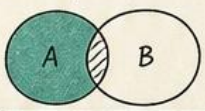
Preserves non-matching rows (puts NULLs on other side).




```
SELECT *
FROM STUDENT s
LEFT JOIN ENROLL e
 ON s.sid = e.sid
  AND e.sem = '2024-1';
```

Filter in WHERE

Filters after join (may remove left-side orphans).




```
SELECT *
FROM STUDENT s
LEFT JOIN ENROLL e
 ON s.sid = e.sid
 WHERE e.sem = '2024-1';
```

☆ For LEFT JOIN, putting conditions on right table in WHERE turns it into INNER JOIN behavior.

TIP: Prefer INNER for matching only; LEFT when you must keep left-side orphans.

☆ Joins reconnect what normalization split. ☆

08 — ADVANCED SQL

1 AGGREGATES



- COUNT(*)
- SUM(col)
- AVG(col)
- MIN(col)
- MAX(col)

2 GROUP BY + HAVING



```
SELECT dept, COUNT(*)
FROM emp
GROUP BY dept
HAVING COUNT(*) > 5;
```

3 SUBQUERIES

(scalar, IN, EXISTS, correlated)



```
SELECT name
FROM emp e
WHERE e.dept_id IN
 (SELECT id FROM dept
 WHERE loc = 'NY');
```

4 CTEs



```
WITH high_earners AS (
  SELECT *
  FROM emp
  WHERE salary > 100000
)
SELECT * FROM high_earners;
```

5 WINDOW FUNCTIONS

ROW_NUMBER, RANK
OVER (PARTITION BY ...)



```
SELECT name, dept, salary,
ROW_NUMBER() OVER
 (PARTITION BY dept
 ORDER BY salary DESC) rn
FROM emp;
```

6 CASE WHEN



```
SELECT name, salary,
 CASE
  WHEN salary >= 100000 THEN 'High'
  WHEN salary >= 50000 THEN 'Med
  ELSE 'Low'
END AS band
FROM emp;
```

7 UNION / UNION ALL / INTERSECT / EXCEPT



```
SELECT id FROM emp_fulltime
UNION
SELECT id FROM emp_parttime;
```

```
SELECT id FROM emp_a
INTERSECT
SELECT id FROM emp_b;
```

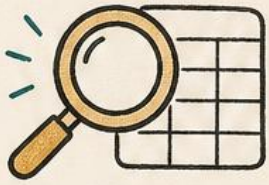
Use **UNION ALL** when duplicates are OK.



TIP: Prefer set-based SQL over row-by-row thinking.

Think in sets. Filter late. Aggregate wisely.

09 — INDEXES



GOAL: Speed up lookups ; trade write cost & space.

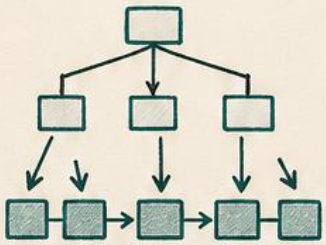
INDEX TYPES

- B-Tree (default)
- Hash
- Bitmap
- Full-text
- Covering index (includes columns to avoid lookups)

CLUSTERED vs NON-CLUSTERED

CLUSTERED INDEX

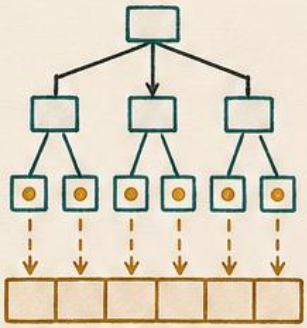
Data is stored in the order of the index.



Leaf nodes = actual data rows (sorted by key)

NON-CLUSTERED INDEX

Index points to data stored elsewhere.



Leaf nodes = pointers to data rows

WHEN INDEXES HELP

- ✓ WHERE filters
- ✓ JOIN keys
- ✓ ORDER BY / GROUP BY
- ✓ Unique constraints

WHEN INDEXES HURT

- ✗ High write volume (more index maintenance)
- ✗ Low selectivity columns (e.g., status, gender)

EXPLAIN / QUERY PLAN TIP

Use EXPLAIN to see how the optimizer runs your query.
Look for Index Scan vs Seq Scan.

Index Scan (good)

```
-> Index Scan using idx_user_email
   on users  (cost=0.29..8.43 rows=1)
   Index Cond: (email = 'a@x.com')
```

✓

Seq Scan (not ideal)

```
-> Seq Scan on users
   (cost=0.00..431.00 rows=10000)
   Filter: (email = 'a@x.com')
```

✗

Index the selective path. Don't index everything.

10 — TRANSACTIONS & ACID

KEY PAGE

A — ATOMICITY



all or nothing

C — CONSISTENCY



rules preserved

I — ISOLATION



concurrent txs don't corrupt

D — DURABILITY



committed data survives crash

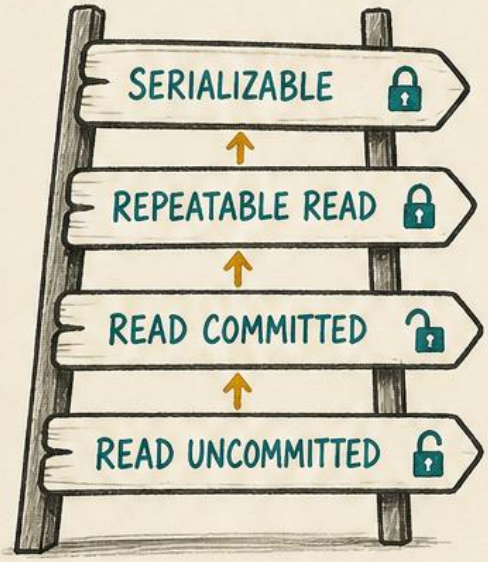
BEGIN ▶ → **COMMIT** ✓ → **ROLLBACK** ↩

start transaction — make changes permanent — undo changes, return to safe state

SAVEPOINT — set a point within a transaction to which you can rollback without aborting the whole tx.



ISOLATION LEVELS (from weak → strong)



	PREVENTS ANOMALIES		
	DIRTY READ	NON-REPEATABLE READ	PHANTOM READS
SERIALIZABLE	✓ Yes	✓ Yes	✓ Yes
REPEATABLE READ	✓ Yes	✓ Yes	✗ No
READ COMMITTED	✓ Yes	✗ No	✗ No
READ UNCOMMITTED	✗ No	✗ No	✗ No

★ *ACID is the contract. Isolation is the dial.* ★

11 — CONCURRENCY CONTROL

GOAL: Correct results when many txs run together.

1 LOCKING

- Shared (S) vs Exclusive (X)

	S	X
S	✓	X
X	X	X

✓ = compatible
X = not compatible

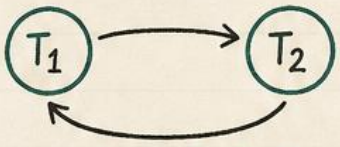
- Two-Phase Locking (2PL)



2PL RULE: Once a tx releases a lock, it cannot acquire any new lock.

2 DEADLOCK

- Wait-for Graph



cycle ⇒ deadlock

- Handling

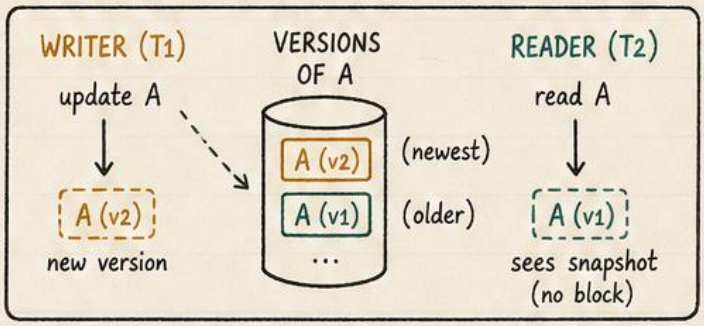
DETECTION
Periodically build WFG, if cycle ⇒ abort a tx.

PREVENTION
- Timeouts
  abort tx if wait too long
- Wound-Wait
  older tx wounds younger; younger waits.



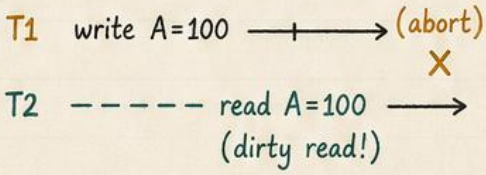
3 MVCC IDEA

Readers don't block writers (versions).

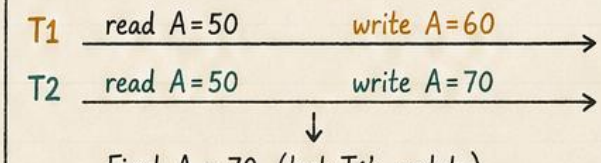


OPTIMISTIC CONCURRENCY
1. Read without locks
2. Do work
3. Validate before commit
   - if no conflict ⇒ commit
   - else ⇒ abort & retry

4 DIRTY READ (BAD)



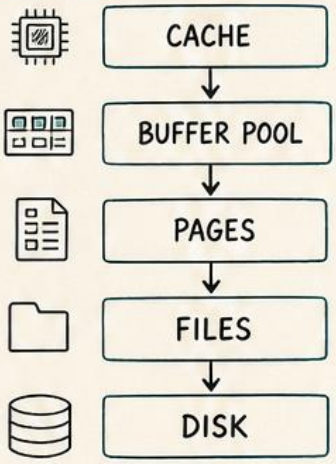
5 LOST UPDATE (BAD)



Locks serialize conflict. MVCC versions history.

12 — STORAGE & FILES

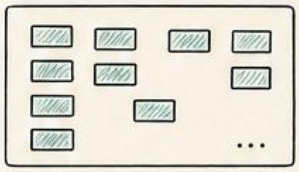
DISK HIERARCHY



FILE ORGANIZATIONS

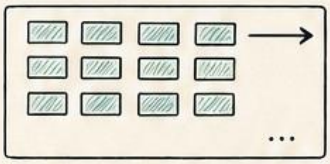
HEAP FILE

- Unordered records
- Insert anywhere
- Simple & fast insert
- Search = scan



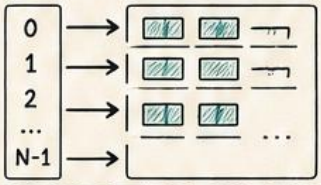
SORTED FILE

- Records sorted on search key
- Efficient range queries
- Costly inserts

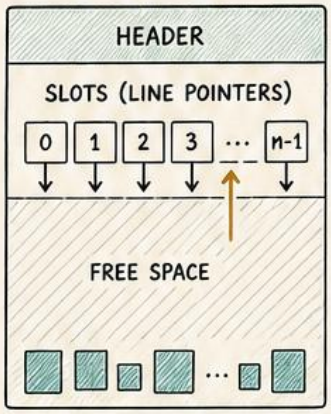


HASH FILE

- Hash on key
- Fast equality search
- Good for point lookups
- Poor for range queries



PAGE LAYOUT



Page header info (page id, LSN, type, etc.)

Slot directory (offsets to records)

Free space grows upwards

Records grow downwards

ROW STORE vs COLUMN STORE

ROW STORE

Store entire rows together

ID	NAME	AGE	CITY
1	ALEX	25	BLR
2	RITA	30	DEL
3	OM	28	MUM
...		...	...

- Good for OLTP
- Fast inserts/updates
- Fetch whole rows

COLUMN STORE

Store columns together

ID	NAME	AGE	CITY
1	ALEX	25	BLR
2	RITA	30	DEL
3	OM	28	MUM
...	...	...	...

- Good for OLAP
- Fast aggregations
- Read only needed cols

WAL / WRITE-AHEAD LOGGING



Log forced to stable storage



RECOVERY

UNDO (ROLLBACK)

- Revert uncommitted changes using log backwards

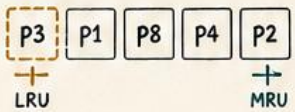
REDO (ROLLFORWARD)

- Reapply committed changes using log forwards

BUFFER POOL POLICIES

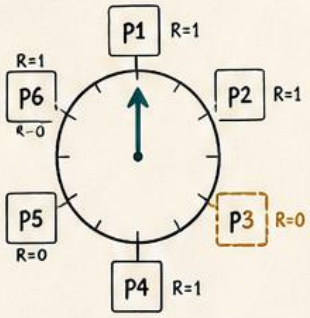
LRU (LEAST RECENTLY USED)

- Evict page that hasn't been used for the longest time



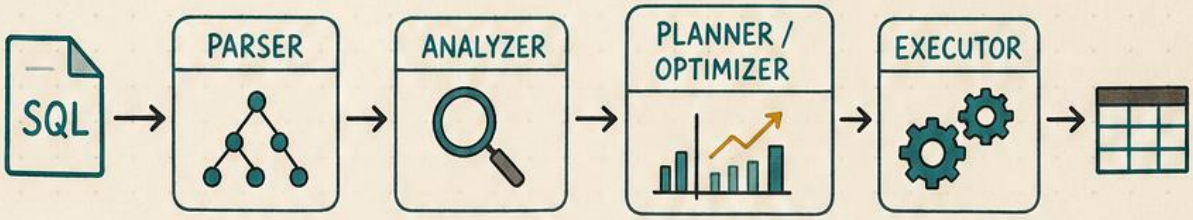
CLOCK (SECOND CHANCE)

- Use reference bit (R)
- Give pages a second chance
- Evict first page with R = 0



★ — Pages in memory. Log first. Then durable disk. — ★

13 — QUERY PROCESSING & OPTIMIZATION



Checks syntax
Builds parse tree

Validates semantics
Resolves names, types, privileges

Explores alternatives
Chooses best plan (lowest cost)

Runs the plan
Returns results

COST-BASED vs RULE-BASED



COST-BASED (preferred)

- Uses statistics (cardinality, histograms, etc.)
- Estimates cost of alternatives
- Picks plan with lowest estimated cost



RULE-BASED

- Uses predefined rules/heuristics
- Faster optimization
- May not find best plan

CARDINALITY ESTIMATES

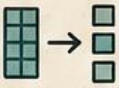


- Estimate number of rows at each step
- Drives cost estimation
- Depends on table stats, histograms, NDV (number of distinct values), data correlation



Bad estimates ⇒ bad plans.
Keep stats up to date!

JOIN ALGORITHMS

ALGORITHM	HOW IT WORKS	WHEN TO USE	COMPLEXITY
NESTED LOOP 	For each row in outer, scan inner table/index	• Small outer table • Index on inner join key • Highly selective predicate	O(N × M) (can be good with indexes)
HASH JOIN 	Build hash table on smaller input, probe with the other	• Large tables • Equality joins (=) • Enough memory for hash	O(N + M) (best for large, unsorted data)
SORT-MERGE JOIN 	Sort both inputs, then merge in order	• Inputs already sorted • Range joins (<, >, BETWEEN) • Large datasets	O(N log N + M log M)

SELECTIVITY TIP



More selective predicates reduce rows early.
Use indexes + filters that cut rows a lot!

EXPLAIN ANALYZE HABIT



Always run and read EXPLAIN (ANALYZE).
Trust the plan, measure the reality.

PUSHDOWN PREDICATES



Apply filters as early as possible (at scan or before join).
Fewer rows → cheaper plans.

PROJECTION PUSHDOWN



Select only needed columns as early as possible.
Less data moved, less to process.

★ —— *Optimizer picks a plan. Stats keep it honest.* —— ★

14 — NOSQL & CAP

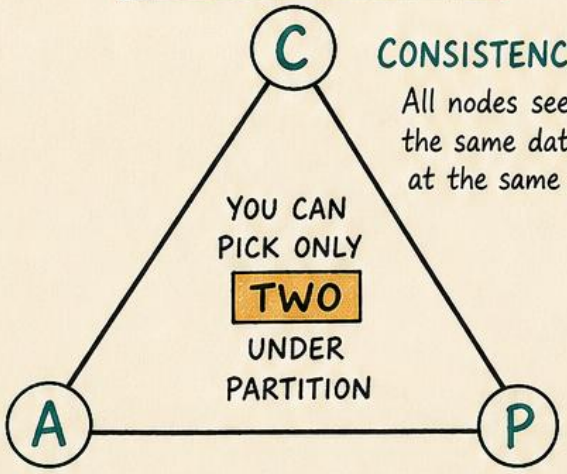
WHEN NOSQL?

- ☑ Flexible schema
- ☑ Massive scale
- ☑ Simple access patterns

TYPES OF NOSQL

DOCUMENT	KEY-VALUE	WIDE-COLUMN	GRAPH
(MongoDB)	(Redis)	(Cassandra)	(Neo4j)

CAP THEOREM



CONSISTENCY
All nodes see the same data at the same time

AVAILABILITY
System remains available and responds to requests

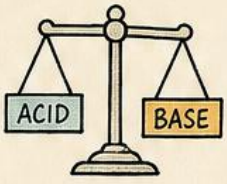
PARTITION TOLERANCE
System continues to operate despite network partitions

CA	CP	AP
Consistency + Availability	Consistency + Partition Tolerance	Availability + Partition Tolerance

BASE vs ACID

NoSQL systems often follow BASE instead of ACID.

- Basically Available
- Soft state
- Eventually consistent



POLYGLOT PERSISTENCE

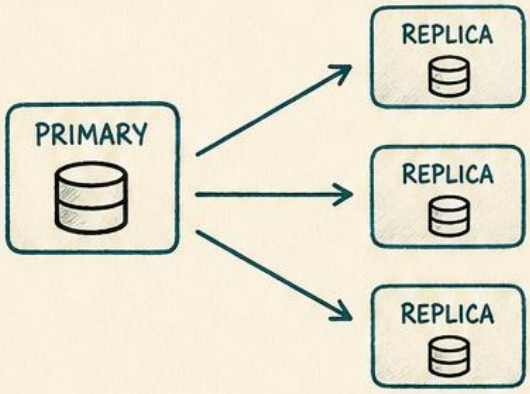
Use the right database for the job.

Combine multiple data stores (SQL, NoSQL, Search, Graph, Cache) to match different access patterns in your system.

SQL for relations. NoSQL for access patterns.

15 — DISTRIBUTED DB BASICS

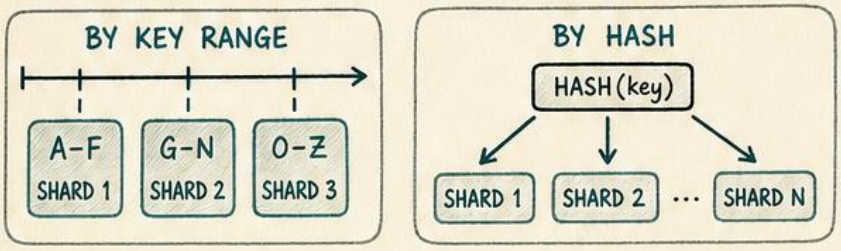
① REPLICATION: PRIMARY-REPLICA



- **SYNC vs ASYNC**
  - **SYNC**: replica ACKs before commit. (stronger consistency, higher latency)
  - **ASYNC**: replica lags behind primary. (lower latency, eventual consistency)
- **FAILOVER**
  If primary fails, a replica is promoted to primary.

② SHARDING / PARTITIONING

Split data across multiple nodes by key range or hash.

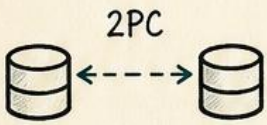


HOTSPOTS

Uneven key distribution can everload a shard.

Monitor & re-balance when needed.

③ DISTRIBUTED TRANSACTIONS



- 2PC (Two-Phase Commit) ensures atomicity across nodes but is **EXPENSIVE**.
- Prefer local transactions + eventual consistency when possible.

WHY EXPENSIVE?

- Extra round trips
- Blocking
- Resource overhead

④ CONSISTENCY MODELS

STRONG
All clients see the same data at the same time. (Linearizable)

EVENTUAL
If no new updates occur, all replicas will converge. (Over time)

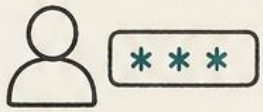
CAUSAL
Preserves the order of causally related events. (Not all orders)

Scale reads with replicas. Scale writes with shards — carefully.

16 — SECURITY & BACKUP

SECURITY

1 AUTHENTICATION



Verify the identity of users.

2 AUTHORIZATION (ROLES/GRANTS)



Define what users can access and do.

3 LEAST PRIVILEGE



Grant only minimum necessary access.

4 ENCRYPTION AT REST & IN TRANSIT



Protect data both stored and in motion.

5 SQL INJECTION PREVENTION (PARAMETERIZED QUERIES)



Use parameterized queries to prevent injections.

6 AUDITING



Log and review activities for accountability.

BACKUP TYPES

FULL



Complete copy of all data.

+ Simple to restore

– Takes more time & storage

INCREMENTAL



Backs up only data changed since last backup.

+ Less storage
+ Faster

– Restore is slower

DIFFERENTIAL



Backs up changes since last full backup.

+ Faster restore

– More storage than incremental

RPO vs RTO

RPO (Recovery Point Objective)

Maximum acceptable amount of data loss measured in time.

RTO (Recovery Time Objective)

Maximum acceptable downtime to restore systems.

POINT-IN-TIME RECOVERY



Use WAL / Binlogs to replay transactions and restore the database to any point in time.

SECURITY & BACKUP CHECKLIST

- [x] Never store plaintext secrets.
- [x] Rotate credentials regularly.
- [x] Test restores periodically.

★ *A backup untested is not a backup.* ★

17 – DESIGN PATTERNS & BEST PRACTICES

PATTERNS & BEST PRACTICES

- Surrogate keys
- Soft deletes (careful)
- Audit columns { created_at, updated_at, created_by, updated_by
- Lookup tables e.g., status, country, role
- Junction tables for M:N 

- Idempotent writes safe to retry
- Optimistic locking version column | `WHERE id = ? AND version = ?`

ANTI-PATTERNS

- ✗ SELECT *
- ✗ God table
- ✗ Missing FKs
- ✗ Over-indexing
- ✗ N+1 queries
- ✗ Storing JSON everything without need

TIP

NAMING CONVENTIONS

- Use singular for tables: user, order
- Use snake_case: created_at, is_active
- Boolean as is_active / has_*
- FK columns as <table>_id

orders	order_items
id (PK)	id (PK)
user_id (FK)	order_id (FK)
status_id (FK)	product_id (FK)
created_at	quantity
is_active	price

MIGRATIONS OVER MANUAL SCHEMA EDITS

Always version your schema changes.
Keep DDL in code. Reproducible. Reviewable. Reliable.

Schema is product.
Evolve it with migrations.

18 — SQL CHEAT SHEET

Copy. Run. Ship.

DDL CREATE ALTER DROP

```
CREATE TABLE t (id INT, name TEXT);
ALTER TABLE t ADD email TEXT;
ALTER TABLE t DROP COLUMN email;
DROP TABLE t;
```

DML INSERT UPDATE DELETE

```
INSERT INTO t (id, name) VALUES (1, 'A');
UPDATE t SET name = 'B' WHERE id = 1;
DELETE FROM t WHERE id = 1;
```

SELECT WHERE ORDER BY LIMIT

```
SELECT * FROM t;
SELECT id, name FROM t WHERE age > 18;
SELECT * FROM t ORDER BY created_at DESC;
SELECT * FROM t LIMIT 10 OFFSET 20;
```

JOINs INNER LEFT

```
SELECT * FROM a
  INNER JOIN b ON a.id = b.a_id;
SELECT * FROM a
  LEFT JOIN b ON a.id = b.a_id;
```

GROUP BY HAVING

```
SELECT dept, COUNT(*) AS cnt
FROM emp
GROUP BY dept;
SELECT dept, COUNT(*) AS cnt
FROM emp
GROUP BY dept HAVING COUNT(*) > 5;
```

AGGREGATES

```
SELECT COUNT(*) FROM t;
SELECT SUM(amount) FROM t;
SELECT AVG(score) FROM t;
SELECT MIN(created_at),
       MAX(created_at) FROM t;
```

SUBQUERY EXISTS

```
SELECT * FROM t1
WHERE EXISTS (
  SELECT 1 FROM t2 WHERE t2.t1_id = t1.id
);
```

CTE WITH

```
WITH recent AS (
  SELECT * FROM orders WHERE created_at >= '2024-01-01'
)
SELECT * FROM recent WHERE status = 'shipped';
```

INDEXES CREATE INDEX

```
CREATE INDEX idx_name ON t(name);
CREATE INDEX idx_email ON t(email);
```

TRANSACTIONS BEGIN COMMIT ROLLBACK

```
BEGIN;
UPDATE t SET balance = balance - 100 WHERE id = 1;
COMMIT;
-- or
ROLLBACK;
```

CONSTRAINTS PK FK UNIQUE CHECK

```
CREATE TABLE t (
  id INT PRIMARY KEY,
  user_id INT,
  email TEXT UNIQUE,
  score INT CHECK (score >= 0),
  CONSTRAINT fk_user FOREIGN KEY (user_id)
      REFERENCES users(id)
);
```

Pro tip:
Read the error.
Check the docs.
Ship it.

Muscle-memory the core SQL verbs.

19 — ROADMAP & MASTER RECAP

LEARNING PATH

1. FUNDAMENTALS & ER
2. RELATIONAL & KEYS
3. NORMALIZATION
4. SQL & JOINS
5. INDEXES & PLANS
6. ACID & CONCURRENCY
7. NOSQL, DISTRIBUTED, SECURITY & PRACTICE

MUST-KNOW INTERVIEW

- Normalization
- Keys / FK
- Joins
- Indexes
- ACID
- Isolation anomalies
- Deadlocks
- CAP

COMMON MISTAKES

- ✗ No normalization
- ✗ Missing indexes on join keys
- ✗ Ignoring transactions
- ✗ SELECT * — SELECT *
- ✗ Untested backups
- ✗ N+1 queries

DESIGN CLEAN. QUERY SMART. KEEP CONSISTENT.